

Recursion

□ Recursive functions

- Functions that call themselves
- Can only solve a base case

□ If not base case

- Break problem into smaller problem(s)
- Launch new copy of function to work on the smaller problem (recursive call/recursive step)
 - Slowly converges towards base case
 - Function makes call to itself inside the return statement
- Eventually base case gets solved
 - Answer works way back up, solves entire problem

Recursion

□ Example: factorial

$$n! = n * (n - 1) * (n - 2) * \dots * 1$$

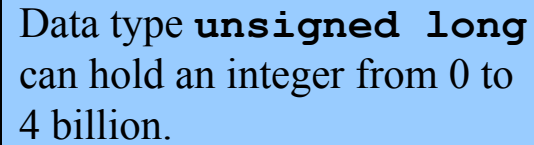
– Recursive relationship ($n! = n * (n - 1)!$)

$$5! = 5 * 4!$$

$$4! = 4 * 3! \dots$$

– Base case ($1! = 0! = 1$)

```
1
2 // Recursive factorial function.
3 #include <iostream>
4
5
6
7
8 #include <iomanip>
9
10 using std::setw;
11
12 unsigned long factorial( unsigned long ); // function prototype
13
14 int main()
15 {
16     // Loop 10 times. During each iteration, calculate
17     // factorial( i ) and display result.
18     for ( int i = 0; i <= 10; i++ )
19         cout << setw( 2 ) << i << "! = "
20             << factorial( i ) << endl;
21
22     return 0; // indicates successful termination
23
24 } // end main
```



Data type **unsigned long**
can hold an integer from 0 to
4 billion.

```
25
26 // recursive definition of function factorial
27 unsigned long factorial( unsigned long number )
28 {
29     // base case
30     if ( number <= 1 )
31         return 1;
32
33     // recursive step
34     else
35         return number * factorial( number - 1 );
36
37 } // end function factorial
```

The base case occurs when we have $0!$ or $1!$. All other cases must be split up (recursive step).



```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Example Using Recursion: Fibonacci Series

□ Fibonacci series: 0, 1, 1, 2, 3, 5, 8...

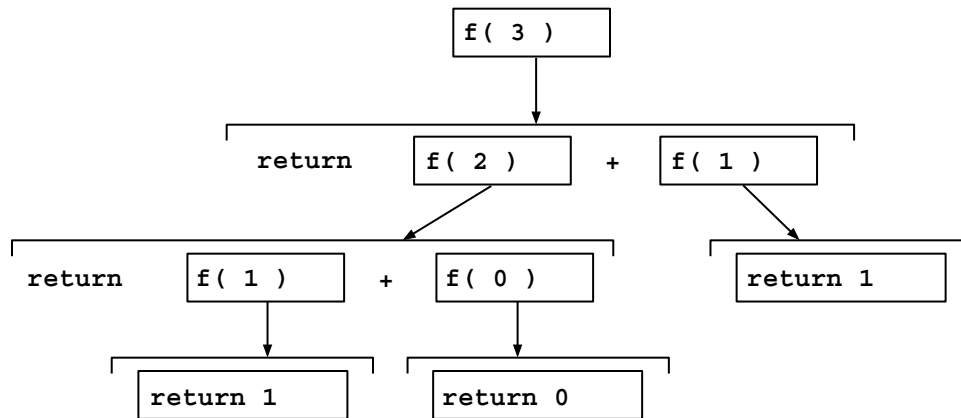
- Each number sum of two previous ones
- Example of a recursive formula:

$$\bullet \text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

□ C++ code for Fibonacci function

```
long fibonacci( long n )
{
    if ( n == 0 || n == 1 )    // base case
        return n;
    else
        return fibonacci( n - 1 ) +
               fibonacci( n - 2 );
}
```

Example Using Recursion: Fibonacci Series



Example Using Recursion: Fibonacci Series

□ Order of operations

- `return fibonacci( n - 1 ) + fibonacci( n - 2 );`

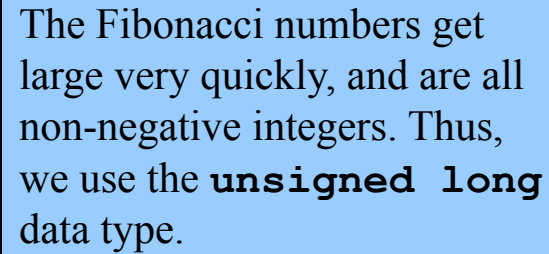
□ Do not know which one executed first

- C++ does not specify
- Only `&&`, `||` and `?:` guaranteed left-to-right evaluation

□ Recursive function calls

- Each level of recursion doubles the number of function calls
 - 30th number = $2^{30} \sim 4$ billion function calls
- Exponential complexity

```
1
2 // Recursive fibonacci function.
3 #include <iostream>
4
5
6
7
8
9 unsigned long fibonacci( unsigned long ); // fun
10
11 int main()
12 {
13     unsigned long result, number;
14
15     // obtain integer from user
16     cout << "Enter an integer: ";
17     cin >> number;
18
19     // calculate fibonacci value for number input by user
20     result = fibonacci( number );
21
22     // display result
23     cout << "Fibonacci(" << number << ") = " << result << endl;
24
25     return 0; // indicates successful termination
```



The Fibonacci numbers get large very quickly, and are all non-negative integers. Thus, we use the **unsigned long** data type.


```
26
27 } // end main
28
29 // recursive definition of function fibonacci
30 unsigned long fibonacci( unsigned long n )
31 {
32     // base case
33     if ( n == 0 || n == 1 )
34         return n;
35
36     // recursive step
37     else
38         return fibonacci( n - 1 ) + fibonacci( n - 2 );
39
40 } // end function fibonacci
```

Enter an integer: 0

Fibonacci(0) = 0

Enter an integer: 1

Fibonacci(1) = 1

Enter an integer: 2

Fibonacci(2) = 1

Enter an integer: 3

Fibonacci(3) = 2

Enter an integer: 4

Fibonacci(4) = 3

Enter an integer: 5

Fibonacci(5) = 5

Enter an integer: 6

Fibonacci(6) = 8

Enter an integer: 10

Fibonacci(10) = 55

Enter an integer: 20

Fibonacci(20) = 6765

Enter an integer: 30

Fibonacci(30) = 832040

Enter an integer: 35

Fibonacci(35) = 9227465

Recursion vs. Iteration

□ Repetition

- Iteration: explicit loop
- Recursion: repeated function calls

□ Termination

- Iteration: loop condition fails
- Recursion: base case recognized

□ Both can have infinite loops

□ Balance between performance (iteration) and good software engineering (recursion)